AI Assisted Coding Lab 14.4

G.Sai teja (2303A52135)

Batch – 41

Question 1: Responsive Homepage Layout for a Startup Website

Prompt:

Generate code for Responsive Homepage Layout for a Startup Website. Design a basic Homepage for a startup company that needs to look good on desktops, tablets and mobile devices. Create a web page structure using HTML that includes a header with navigation links, a main content section, a footer with basic information. Generate a responsive CSS. Ensure the layout adjusts smoothly across different screen sizes. Generate a clean and well structured code.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>StartupHub - Innovative Solutions for Modern Business</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        :root {
```

```
    --primary-color: #2563eb;
    --secondary-color: #1e40af;
    --accent-color: #f59e0b;
    --text-color: #1f2937;
    --light-text: #6b7280;
    --light-bg: #f9fafb;
    --white: #ffffff;
    --shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    --shadow-lg: 0 10px 15px rgba(0, 0, 0, 0.15);
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    color: var(--text-color);
    line-height: 1.6;
    background-color: var(--white);
}

/* Header and Navigation */
header {
    background-color: var(--white);
    box-shadow: var(--shadow);
    position: sticky;
    top: 0;
    z-index: 100;
}
```

```
nav {
    display: flex;
    justify-content:  space-between;
    align-items: center;
    padding:  1rem 2rem;
    max-width:   1200px;
    margin: 0 auto;
}

.logo {
    font-size: 1.5rem;
    font-weight: bold;
    color: var(--primary-color);
    text-decoration: none;
}

.nav-links
    { display: flex;
    list-style: none;
    gap: 2rem;
}

.nav-links a {
    text-decoration: none;
    color: var(--text-color);
    font-weight: 500;
    transition: color 0.3s ease;
```

```
}

.nav-links a:hover {
    color: var(--primary-color);
}

.cta-button {
    background-color: var(--primary-color);
    color: var(--white);
    padding: 0.75rem 1.5rem;
    border-radius: 5px;
    text-decoration: none;
    transition: background-color 0.3s ease;
    border: none;
    cursor: pointer;
}

.cta-button:hover {
    background-color: var(--secondary-color);
}

.hamburger
    { display: none;
    flex-direction: column;
    cursor: pointer;
}
```

```
.hamburger span
    { width: 25px;
    height: 3px;
    background-color: var(--text-color);
    margin: 5px 0;
    transition: 0.3s;
}

/* Hero Section */
.hero {
    background: linear-gradient(135deg, var(--primary-color), var(--
secondary-color));
    color: var(--white);
    padding: 4rem 2rem;
    text-align: center;
    min-height: 500px;
    display: flex;
    flex-direction:  column;
    justify-content:  center;
    align-items: center;
}

.hero-content {
    max-width: 600px;
    margin: 0 auto;
}

.hero h1 {
```

```
    font-size: 3rem;
    margin-bottom: 1rem;
    font-weight: 700;
}

.hero p {
    font-size: 1.25rem;
    margin-bottom: 2rem;
    opacity: 0.9;
}

.hero .cta-button
    { display: inline-block;
    background-color: var(--accent-color);
    font-size: 1.1rem;
    padding: 1rem 2rem;
}

.hero .cta-button:hover
    { background-color:
    #d97706;
}

/* Main Content */
main {
    max-width: 1200px;
    margin: 0 auto;
    padding: 3rem 2rem;
```

```
}

/* Features Section */
.features {  margin:
    3rem 0;
}

.section-title {
    font-size: 2.5rem;
    text-align: center;
    margin-bottom: 3rem;
    color: var(--text-color);
}

.features-grid
    { display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 2rem;
}

.feature-card {
    background-color: var(--light-bg);
    padding: 2rem;
    border-radius: 8px;
    text-align: center;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
    cursor: pointer;
```

```
}

.feature-card:hover
   { transform: translateY(-
   5px);
   box-shadow: var(--shadow-lg);
}

.feature-icon
   { font-size:
   3rem;
   margin-bottom: 1rem;
}

.feature-card h3
   { font-size:
   1.5rem;
   margin-bottom: 1rem; color:
   var(--primary-color);
}

.feature-card p {
   color: var(--light-text);
   font-size: 0.95rem;
}

/* About Section */
.about {
   background-color: var(--light-bg);
   padding: 3rem 2rem;
```

```
    margin: 3rem -2rem;
}

.about-content {
    max-width: 1200px;
    margin: 0 auto;
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 3rem;
    align-items: center;
}

.about h2 {
    font-size: 2rem;
    margin-bottom: 1rem;
    color: var(--text-color);
}

.about p {
    color: var(--light-text);
    margin-bottom: 1rem;
    line-height: 1.8;
}

/* Footer */
footer {
    background-color: var(--text-color);
```

```
    color: var(--white);
    padding: 3rem 2rem;
}

.footer-content {
    max-width: 1200px;
    margin: 0 auto;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 2rem;
    margin-bottom: 2rem;
}

.footer-section h4
    { margin-bottom:
    1rem;
    color: var(--accent-color);
}

.footer-section ul
    { list-style: none;
}

.footer-section ul li
    { margin-bottom:
    0.5rem;
}

.footer-section a {
```

```
    color: #d1d5db;
    text-decoration: none;
    transition: color 0.3s ease;
}

.footer-section a:hover
    { color: var(--accent-color);
}

.footer-bottom {
    border-top: 1px solid #4b5563;
    padding-top: 2rem;
    text-align: center;
    color: #9ca3af;
}

/* Mobile Responsive */
@media (max-width: 768px) {
    .nav-links
        { display: none;
        position: absolute;
        top: 70px;
        left: 0;
        flex-direction: column;
        background-color: var(--white);
        width: 100%;
        text-align: center;
```

```
    gap: 0;
}

.nav-links.active
    { display: flex;
}

.nav-links li
    { padding: 1rem 0;
    border-bottom: 1px solid var(--light-bg);
}

.hamburger
    { display: flex;
}

.hero h1 {
    font-size: 2rem;
}

.hero p {
    font-size: 1rem;
}

.section-title {
    font-size: 1.75rem;
    margin-bottom: 2rem;
```

```
    }

    .features-grid {
        grid-template-columns: 1fr;
        gap: 1.5rem;
    }

    .about-content {
        grid-template-columns: 1fr;
        gap: 2rem;
    }

    nav {
        padding: 1rem;
    }

    main {
        padding: 2rem 1rem;
    }

    .about {
        margin: 2rem -1rem;
        padding: 2rem 1rem;
    }
}

@media (max-width: 480px) {
```

```
.logo {
    font-size: 1.25rem;
}

.hero {
    padding: 2rem 1rem;
    min-height: 350px;
}

.hero h1 {
    font-size: 1.5rem;
}

.hero p {
    font-size: 0.9rem;
}

.section-title {
    font-size: 1.5rem;
}

.feature-icon {
    font-size: 2.5rem;
}

.footer-content {
    grid-template-columns: 1fr;
```

```
            text-align: center;
        }

        .footer-section ul
            { margin-bottom:
            1rem;
        }
    }
  </style>
</head>
<body>
  <!-- Header and Navigation -->
  <header>
    <nav>
      <a href="#" class="logo">StartupHub</a>
      <ul class="nav-links" id="navLinks">
        <li><a href="#features">Features</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#pricing">Pricing</a></li>
        <li><a href="#contact">Contact</a></li>
        <li><a href="#" class="cta-button">Get Started</a></li>
      </ul>
      <div class="hamburger" id="hamburger">
        <span></span>
        <span></span>
        <span></span>
      </div>
    </nav>
```

```
    </header>


    <!-- Hero Section -->
    <section class="hero">
        <div class="hero-content">
            <h1>Welcome to StartupHub</h1>
            <p>Build, launch, and scale your business with our innovative
solutions</p>
            <a href="#" class="cta-button">Start Your Journey</a>
        </div>
    </section>


    <!-- Main Content -->
    <main>
        <!-- Features Section -->
        <section id="features" class="features">
            <h2 class="section-title">Our Features</h2>
            <div class="features-grid">
                <div class="feature-card">
                    <div                class="feature-icon">
                    <h3>Fast Performance</h3>
                    <p>Lightning-fast solutions optimized for speed and efficiency to
accelerate your business growth.</p>
                </div>
                <div class="feature-card">
                    <div  class="feature-icon">  </div>
                    <h3>Secure & Reliable</h3>
```

Enterprise-grade security with 99.9% uptime guarantee to protect your valuable data.

<h3>Analytics Dashboard</h3>

Real-time insights and comprehensive analytics to make data-driven decisions.

<h3>24/7 Support</h3>

Dedicated support team available round-the-clock to assist you whenever needed.

<h3>Easy Integration</h3>

Seamless integration with your existing tools and platforms in minutes.

9

<h3>Innovation First</h3>

Cutting-edge technology and continuous innovation to keep you ahead of competition.

</section>

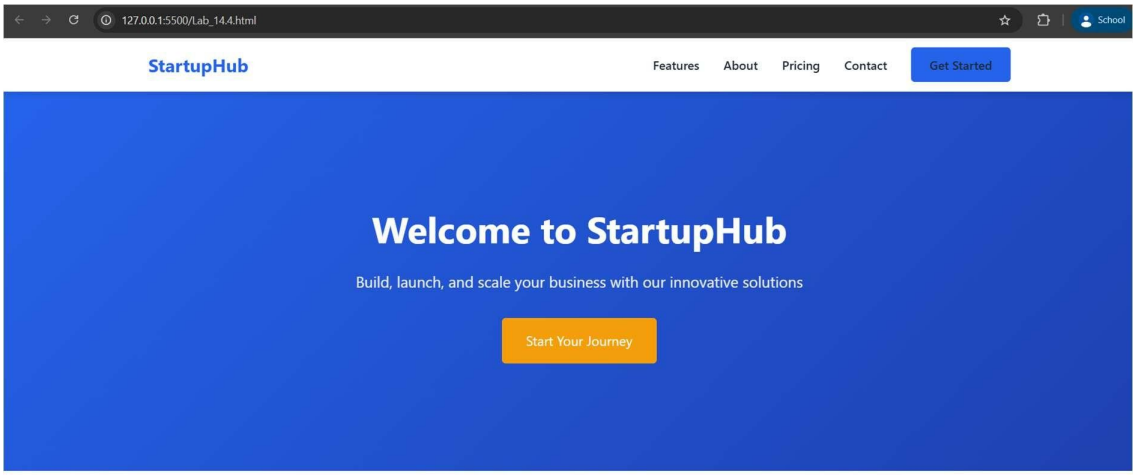
```
    <!-- About Section -->
    <section id="about" class="about">
      <div class="about-content">
        <div>
          <h2>About StartupHub</h2>
          <p>StartupHub is dedicated to empowering entrepreneurs and small businesses with the tools and resources they need to succeed in today's digital landscape.</p>
          <p>We understand the unique challenges faced by startups and have designed our platform to be flexible, scalable, and user-friendly. Our mission is to democratize access to enterprise-level solutions for businesses of all sizes.</p>
        </div>
        <div>
          <h2>Why Choose Us?</h2>
          <p><strong>Affordability:</strong> Transparent pricing with no hidden fees, designed for startups and growing businesses.</p>
          <p><strong>Scalability:</strong> Grow your business without worrying about infrastructure limitations.</p>
          <p><strong>Community:</strong> Join thousands of entrepreneurs and access our vibrant community of innovators.</p>
        </div>
      </div>
    </section>
  </main>

  <!-- Footer -->
  <footer>
    <div class="footer-content">
```

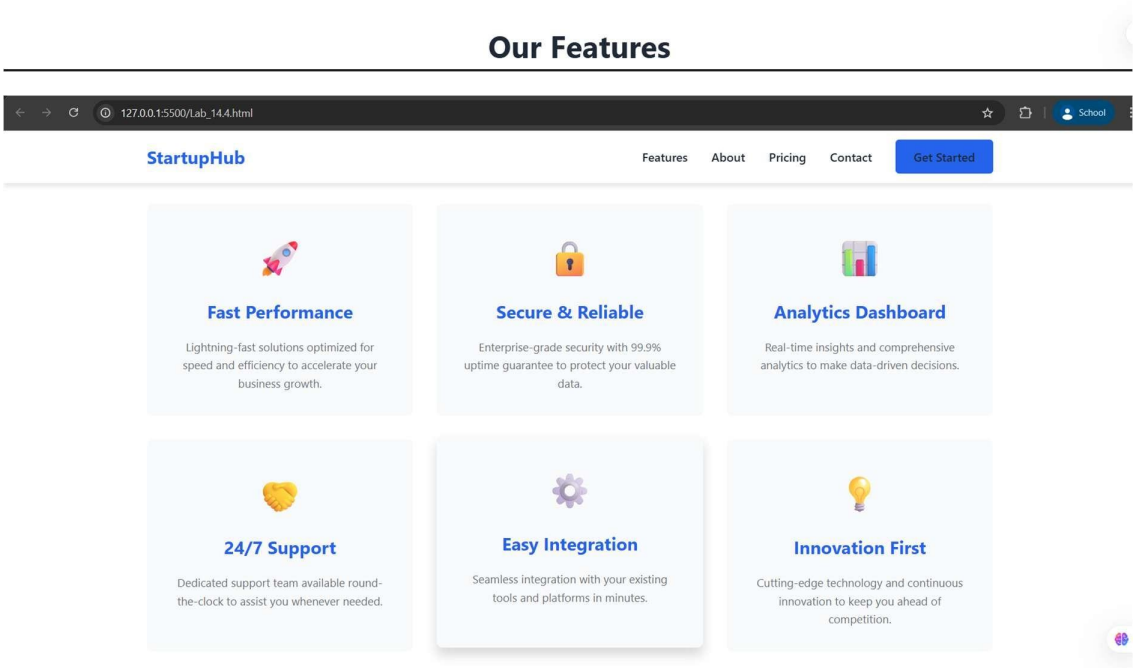
```
<div class="footer-section">
    <h4>Product</h4>
    <ul>
        <li><a href="#">Features</a></li>
        <li><a href="#">Pricing</a></li>
        <li><a href="#">Security</a></li>
        <li><a href="#">Roadmap</a></li>
    </ul>
</div>
<div class="footer-section">
    <h4>Company</h4>
    <ul>
        <li><a href="#">About</a></li>
        <li><a href="#">Blog</a></li>
        <li><a href="#">Careers</a></li>
        <li><a href="#">Contact</a></li>
    </ul>
</div>
<div class="footer-section">
    <h4>Resources</h4>
    <ul>
        <li><a href="#">Documentation</a></li>
        <li><a href="#">API Reference</a></li>
        <li><a href="#">Support</a></li>
        <li><a href="#">Community</a></li>
    </ul>
</div>
```

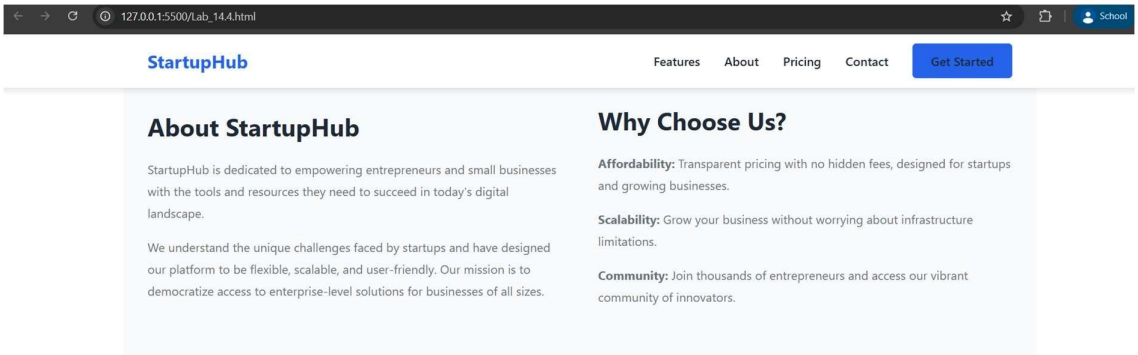
```
            <div class="footer-section">
                <h4>Legal</h4>
                <ul>
                    <li><a href="#">Privacy Policy</a></li>
                    <li><a href="#">Terms of Service</a></li>
                    <li><a href="#">Cookie Policy</a></li>
                    <li><a href="#">GDPR</a></li>
                </ul>
            </div>
        </div>
        <div class="footer-bottom">
            <p>&copy; 2024 StartupHub. All rights reserved. | Designed with 
for entrepreneurs</p>
        </div>
    </footer>

    <script>
        // Mobile Menu Toggle
        const hamburger = document.getElementById('hamburger');
        const navLinks = document.getElementById('navLinks');

        hamburger.addEventListener('click', () =>
            { navLinks.classList.toggle('active');
        });

        // Close menu when link is clicked
        document.querySelectorAll('.nav-links a').forEach(link => {
            link.addEventListener('click', () => {
```

```
            navLinks.classList.remove('active');
        });
    });

    // Smooth scroll behavior for anchor links
    document.querySelectorAll('a[href^="#"]').forEach(anchor => {
        anchor.addEventListener('click', function (e)
            { e.preventDefault();
            const target = document.querySelector(this.getAttribute('href'));
            if (target) {
                target.scrollIntoView({ beh
                    avior: 'smooth', block:
                    'start'
                });
            }
        });
    });
  </script>
</body>
</html>
```

Output:


127.0.0.1:5500/Lab_14.4.html
School
StartupHub
Features
About
Pricing
Contact
Get Started
Welcome to StartupHub
Build, launch, and scale your business with our innovative solutions
Start Your Journey


Our Features
127.0.0.1:5500/Lab_14.4.html
School
StartupHub
Features
About
Pricing
Contact
Get Started
Fast Performance
Lightning-fast solutions optimized for speed and efficiency to accelerate your business growth.
Secure & Reliable
Enterprise-grade security with 99.9% uptime guarantee to protect your valuable data.
Analytics Dashboard
Real-time insights and comprehensive analytics to make data-driven decisions.
24/7 Support
Dedicated support team available round-the-clock to assist you whenever needed.
Easy Integration
Seamless integration with your existing tools and platforms in minutes.
Innovation First
Cutting-edge technology and continuous innovation to keep you ahead of competition.



Explanation:

This website code creates a startup company homepage that looks great on phones, tablets, and computers. The top part has a navigation menu that turns into a hamburger button on small screens, and the rest has colorful sections showing features, company info, and links at the bottom. When you click menu items, the page smoothly scrolls to different sections, and all the text and boxes automatically resize to fit any screen size perfectly.

Question 2: Interactive Call-to-Action Button

Prompt:

Generate a code for Interactive Call-to-Action Button. Design a marketing page that requires call-to-action button that responds when users interact with it. Add a button to web page using HTML. Generate a javascript that responds to the button click by displaying a message or alert. Add meaningful comments explaining the javascript logic. Generate a readable and commented code. Occursion of page reload shouldn't occur.

Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Interactive CTA Button - Marketing Page</title>
<style>
    * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
    }

    body {
        font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        min-height: 100vh;
        display: flex;
        justify-content: center;
        align-items: center;
        padding: 20px;
    }

    .container {
        background-color: white;
        border-radius: 10px;
        box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
        padding: 50px;
        max-width: 600px;
        width: 100%;
    }
```

```
h1 {
    color: #333;
    margin-bottom: 15px;
    text-align: center;
}

.subtitle
    { color:
    #666;
    text-align: center;
    margin-bottom: 30px;
    font-size: 1.1rem;
}

.content {
    margin-bottom: 40px;
    line-height: 1.6;
    color: #555;
}

.cta-section
    { display: flex;
    flex-direction: column;
    gap: 15px;
    margin-bottom: 30px;
}
```

```
/* Primary CTA Button */
.btn-primary {
    background-color: #667eea;
    color: white;
    padding: 15px 30px;
    border: none;
    border-radius: 5px;
    font-size: 1.1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
}

.btn-primary:hover
    { background-color:
    #5568d3; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

.btn-primary:active
    { transform: translateY(0);
}

/* Secondary CTA Button */
.btn-secondary {
    background-color: #764ba2;
    color: white;
```

```
    padding: 15px 30px;
    border: none;
    border-radius: 5px;
    font-size: 1.1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
}

.btn-secondary:hover
    { background-color:
    #63408a; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(118, 75, 162, 0.4);
}

.btn-secondary:active
    { transform: translateY(0);
}

/* Tertiary CTA Button */
.btn-tertiary {
    background-color: #f39c12;
    color: white;
    padding: 15px 30px;
    border: none;
    border-radius: 5px;
    font-size: 1.1rem;
```

```
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
}

.btn-tertiary:hover
    { background-color:
    #d68910; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(243, 156, 18, 0.4);
}

.btn-tertiary:active
    { transform: translateY(0);
}

/* Message Display Area */
.message-container
    { background-color: #f0f0f0;
    border-left: 4px solid #667eea;
    padding: 15px;
    border-radius: 5px;
    margin-top: 20px;
    display: none;
}

.message-container.show
    { display: block;
```

```
    animation: slideIn 0.3s ease;
}

.message-container.success
    { background-color:
    #d4edda; border-left-color:
    #28a745;
}

.message-container.info
    { background-color:
    #d1ecf1; border-left-color:
    #17a2b8;
}

.message-container.warning
    {           background-color:
    #fff3cd;    border-left-color:
    #ffc107;
}

.message-text
    { color: #333;
    font-weight: 500;
}

/* Modal Popup */
.modal { display:
    none;
    position: fixed;
```

```
    z-index: 1000;
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    animation: fadeIn 0.3s ease;
}

.modal.show
    { display: flex;
    justify-content: center;
    align-items: center;
}

.modal-content
    { background-color:
    white; padding: 30px;
    border-radius: 10px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
    max-width: 400px;
    text-align: center;
    animation: slideUp 0.3s ease;
}

.modal-content h2
    { color: #333;
```

```
    margin-bottom: 15px;
}

.modal-content p
    { color: #666;
    margin-bottom: 20px;
}

.close-btn {
    background-color: #667eea;
    color: white;
    padding: 10px 25px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-weight: bold;
    transition: background-color 0.3s ease;
}

.close-btn:hover {
    background-color: #5568d3;
}

@keyframes slideIn
    { from {
        transform: translateX(-20px);
        opacity: 0;
```

```
    }
    to {
        transform: translateX(0);
        opacity: 1;
    }
}

@keyframes fadeIn
    { from {
        opacity: 0;
    }
    to {
        opacity: 1;
    }
}

@keyframes slideUp
    { from {
        transform: translateY(20px);
        opacity: 0;
    }
    to {
        transform: translateY(0);
        opacity: 1;
    }
}
```

```
        .click-count {
            text-align: center;
            color: #667eea;
            font-weight: bold;
            margin-top: 20px;
        }

        @media (max-width: 600px) {
            .container
                { padding:
                30px;
            }

            h1 {
                font-size: 1.5rem;
            }

            .btn-primary, .btn-secondary, .btn-tertiary
                { padding: 12px 20px;
                font-size: 1rem;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <!-- Page Heading -->
        <h1>Unlock Your Potential</h1>
```

```
    <p class="subtitle">Experience the power of our premium service
today</p>


    <!-- Main Content -->
    <div class="content">
        <p>Join thousands of satisfied customers who have transformed their
business using our innovative solutions. Don't miss out on this opportunity to
revolutionize your workflow and achieve unprecedented success.</p>
    </div>


    <!-- Call-to-Action Section with Multiple Buttons -->
    <div class="cta-section">
        <!-- Primary CTA Button -->
        <button class="btn-primary" id="ctaButton1">
            Get Started Now
        </button>

        <!-- Secondary CTA Button -->
        <button class="btn-secondary" id="ctaButton2">
            Learn More
        </button>

        <!-- Tertiary CTA Button -->
        <button class="btn-tertiary" id="ctaButton3">
            Contact Us
        </button>
    </div>
```

```
        <!-- Message Display Area -->
        <div class="message-container" id="messageContainer">
            <p class="message-text" id="messageText">Your message will appear here</p>
        </div>

        <!-- Click Counter Display -->
        <div class="click-count" id="clickCount" style="display: none;">
            You have clicked buttons <span id="totalClicks">0</span> times
        </div>
    </div>

    <!-- Modal Popup -->
    <div class="modal" id="modal">
        <div class="modal-content">
            <h2 id="modalTitle">Success!</h2>
            <p id="modalMessage">Your request has been received successfully.</p>
            <button class="close-btn" id="closeModal">Close</button>
        </div>
    </div>

    <script>
        // ==========================================
        // JavaScript: Interactive CTA Button Handling
        // ==========================================

        // Get references to all button elements using getElementById
```

```
const ctaButton1 = document.getElementById('ctaButton1');
const ctaButton2 = document.getElementById('ctaButton2');
const ctaButton3 = document.getElementById('ctaButton3');

// Get references to message and modal elements
const messageContainer = document.getElementById('messageContainer');
const messageText = document.getElementById('messageText');
const modal = document.getElementById('modal');
const modalTitle = document.getElementById('modalTitle');
const modalMessage = document.getElementById('modalMessage');
const closeModal = document.getElementById('closeModal');
const clickCount = document.getElementById('clickCount');
const totalClicks = document.getElementById('totalClicks');

// Variable to track total button clicks
let clickCounter = 0;

// ==========================================
// Function: Display inline message
// Parameters: message (string), type (success/info/warning)
// Returns: None
// ==========================================
function displayMessage(message, type = 'info') {
    // Update the message text content
    messageText.textContent = message;

    // Remove all existing type classes
```

```
    messageContainer.classList.remove('success', 'info', 'warning');

    // Add the specified type class for styling
    messageContainer.classList.add(type);

    // Show the message container by adding 'show' class
    messageContainer.classList.add('show');

    // Auto-hide the message after 4 seconds
    setTimeout(() => {
        messageContainer.classList.remove('show');
    }, 4000);
}

// ==========================================
// Function: Display modal popup dialog
// Parameters: title (string), message (string)
// Returns: None
// ==========================================
function showModal(title, message) {
    // Set the modal title and message content
    modalTitle.textContent = title;
    modalMessage.textContent = message;

    // Show the modal by adding 'show' class
    modal.classList.add('show');
}
```

```
// ==============================================
// Function: Hide modal popup dialog
// Returns: None
// ==============================================
function hideModal() {
    // Hide the modal by removing 'show' class
    modal.classList.remove('show');
}

// ==============================================
// Function: Update and display click counter
// Returns: None
// ==============================================
function updateClickCounter() {
    // Increment the click counter by 1
    clickCounter++;

    // Update the displayed total clicks
    totalClicks.textContent = clickCounter;

    // Show the click count section
    clickCount.style.display = 'block';
}

// ==============================================
// Event Listener: Button 1 - "Get Started Now"
```

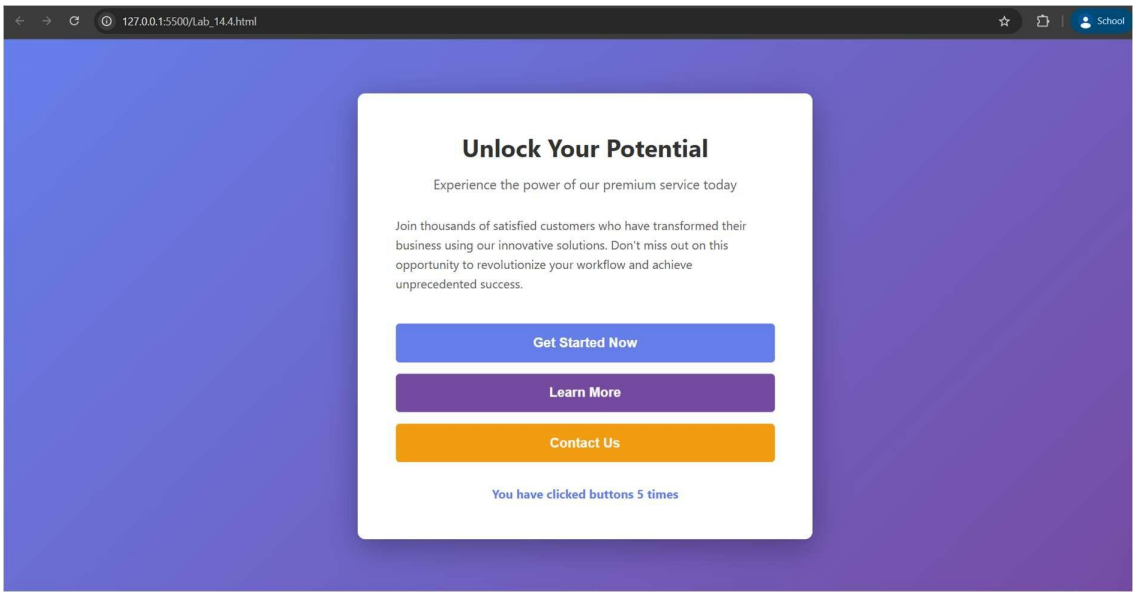
```
// Action: Displays a modal popup and inline message
// ==============================================
ctaButton1.addEventListener('click', function() {
    // Call function to display modal with success message
    showModal(
        'Welcome!',
        'Thank you for your interest! We\'re excited to have you on board. Check your email for next steps.'
    );

    // Display inline confirmation message
    displayMessage('✓ Your request has been submitted successfully!', 'success');

    // Update the click counter
    updateClickCounter();
});

// ==============================================
// Event Listener: Button 2 - "Learn More"
// Action: Displays inline informational message
// ==============================================
ctaButton2.addEventListener('click', function() {
    // Display informational message
    displayMessage(
        'ℹ Learn more: Our service includes 24/7 support, advanced analytics, and unlimited scalability. Visit our documentation for details.',
        'info'
```

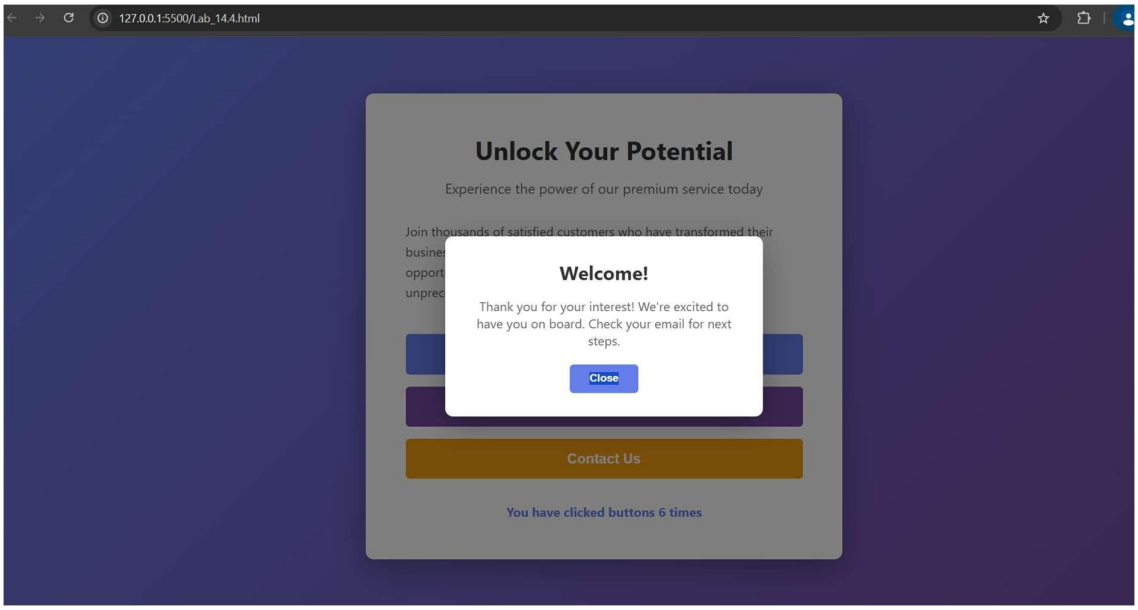
```
    );

    // Update the click counter
    updateClickCounter();
});

// ==========================================
// Event Listener: Button 3 - "Contact Us"
// Action: Displays modal and warning message
// ==========================================
ctaButton3.addEventListener('click', function() {
    // Call function to display modal with contact confirmation
    showModal(
        'Contact Request Sent!',
        'Our team will get back to you within 24 hours. We look forward to connecting with you!'
    );

    // Display warning type message
    displayMessage(
        '⚠ A representative will contact you soon. Please check your phone and email regularly.',
        'warning'
    );

    // Update the click counter
    updateClickCounter();
});
```

```
// ==============================================
// Event Listener: Close Modal Button
// Action: Closes the modal popup when clicked
// ==============================================
closeModal.addEventListener('click', function() {
    // Call function to hide modal
    hideModal();
});

// ==============================================
// Event Listener: Modal Background Click
// Action: Closes modal when clicking outside the modal content
// Purpose: Better UX - allows closing by clicking backdrop
// ==============================================
modal.addEventListener('click', function(event) {
    // Check if the click occurred on the modal backdrop (not the content)
    if (event.target === modal) {
        hideModal();
    }
});

// ==============================================
// Code Summary:
// - Buttons trigger different response types
// - Messages auto-dismiss after 4 seconds
// - No page reload occurs - pure DOM manipulation
```

```
        // - Modals can be closed by button or backdrop click
        // - Click counter tracks user interactions
        // - All animations are CSS-based for smooth UX
        // ==========================================
    </script>
</body>
</html>
```

Output:





Explanation:

This page has three colorful buttons that respond instantly when you click them without reloading the web page. Each button shows different messages—some pop up in a dialog box and some appear as notifications below the buttons, with emojis and special styling to show if it's good news, information, or a warning. The page also keeps count of how many times you've clicked the buttons, and all the messages disappear automatically after a few seconds.

Question 3: Contact Form with Client-Side Validation

Prompt:

Generate code for Contact Form with Client-Slide Validation. Develop a contact form for a company website to collect user queries safely and correctly. Design a form with fields for Name,Email, Message. Generate a javascript validation logic that prevents empty submissions, checks for a valid email format, display clear error messages for invalid input. Invalid input should show inline error messages, valid input should display a success message. Validation should happen on the client side using javascript.

Code:

```
<!DOCTYPE html>

<html lang="en">

<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Contact Us - Company Website</title>
<style>
    * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
    }

    body {
        font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        min-height: 100vh;
        display: flex;
        justify-content: center;
        align-items: center;
        padding: 20px;
    }

    .container {
        background-color: white;
        border-radius: 10px;
        box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
        padding: 50px;
        max-width: 600px;
        width: 100%;
```

```
}

.header {
    text-align: center;
    margin-bottom: 40px;
}

h1 {
    color: #333;
    margin-bottom: 10px;
    font-size: 2.5rem;
}

.subtitle
    { color:
    #666;
    font-size: 1.1rem;
}

/* Form Styling */
.form-group {
    margin-bottom: 25px;
}

label {
    display: block;
    margin-bottom: 8px;
    color: #333;
```

```
    font-weight: 600;
    font-size: 1rem;
}

input[type="text"],
input[type="email"],
textarea {
    width: 100%;
    padding: 12px 15px;
    border: 2px solid #e0e0e0;
    border-radius: 5px;
    font-size: 1rem;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    transition: border-color 0.3s ease, box-shadow 0.3s ease;
}

input[type="text"]:focus,
input[type="email"]:focus,
textarea:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 8px rgba(102, 126, 234, 0.3);
}

/* Input field with error state */
input.error,
textarea.error {
```

```
    border-color: #dc3545;
    background-color: #fff5f5;
}

/* Input field with success state */
input.success,
textarea.success {
    border-color: #28a745;
    background-color: #f0fdf4;
}

textarea {
    resize: vertical;
    min-height: 150px;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

/* Error Message Styling */
.error-message
    { color: #dc3545;
    font-size: 0.9rem;
    margin-top: 5px;
    display: none;
    animation: slideDown 0.3s ease;
}

.error-message.show {
```

```
    display: block;
}

.error-message::before
    { content: "╳ ";
    font-weight: bold;
}

/* Success Message Styling */
.success-message
    { color: #28a745;
    font-size: 0.9rem;
    margin-top: 5px;
    display: none;
    animation: slideDown 0.3s ease;
}

.success-message.show
    { display: block;
}

.success-message::before
    { content: "✓ ";
    font-weight: bold;
}

/* Form Submit Button */
```

```
.btn-submit
    { width: 100%;
    padding: 15px;
    background-color: #667eea;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 1.1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
    margin-top: 20px;
}

.btn-submit:hover
    { background-color:
    #5568d3; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

.btn-submit:active
    { transform: translateY(0);
}

.btn-submit:disabled
    { background-color:
    #ccc; cursor: not-
    allowed;
```

```
    transform: none;
}

/* Overall Form Success Container */
.form-success {
    background-color: #d4edda;
    border-left: 4px solid #28a745;
    padding: 15px;
    border-radius: 5px;
    display: none;
    margin-bottom: 20px;
    animation: slideDown 0.3s ease;
}

.form-success.show
    { display: block;
}

.form-success h3
    { color: #155724;
    margin-bottom: 8px;
}

.form-success p
    { color: #155724;
    margin: 0;
}
```

```
/* Animations */
@keyframes slideDown {
    from {
        transform: translateY(-10px);
        opacity: 0;
    }
    to {
        transform: translateY(0);
        opacity: 1;
    }
}

/* Reset Button */
.btn-reset
    { width: 100%;
    padding: 12px;
    background-color: #6c757d;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
    margin-top: 10px;
}
```

```
.btn-reset:hover {
    background-color: #5a6268;
}

/* Required field indicator */
.required {
    color: #dc3545;
    font-weight: bold;
}

/* Responsive Design */
@media (max-width: 600px) {
    .container
        { padding:
        30px;
    }

    h1 {
        font-size: 2rem;
    }

    .form-group {
        margin-bottom: 20px;
    }

    input[type="text"],
    input[type="email"],
```

```
            textarea {
                padding: 10px 12px;
                font-size: 16px; /* Prevents zoom on iOS */
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <!-- Header Section -->
        <div class="header">
            <h1>Contact Us</h1>
            <p class="subtitle">We'd love to hear from you. Send us a
message!</p>
        </div>

        <!-- Form Success Message -->
        <div class="form-success" id="formSuccess">
            <h3>Thank You!</h3>
            <p>Your message has been sent successfully. We'll get back to you
soon.</p>
        </div>

        <!-- Contact Form -->
        <form id="contactForm" novalidate>
            <!-- Name Field Group -->
            <div class="form-group">
                <label for="nameField">
```

```
        Full Name <span class="required">*</span>
    </label>
    <input
        type="text"
        id="nameField"
        name="name"
        placeholder="Enter your full name"
    >
    <!-- Error message container for name field -->
    <div class="error-message" id="nameError"></div>
    <!-- Success message container for name field -->
    <div class="success-message" id="nameSuccess"></div>
</div>

<!-- Email Field Group -->
<div class="form-group">
    <label for="emailField">
        Email Address <span class="required">*</span>
    </label>
    <input
        type="email"
        id="emailField"
        name="email"
        placeholder="Enter your email address"
    >
    <!-- Error message container for email field -->
    <div class="error-message" id="emailError"></div>
```

```
            <!-- Success message container for email field -->
            <div class="success-message" id="emailSuccess"></div>
        </div>

        <!-- Message Field Group -->
        <div class="form-group">
            <label for="messageField">
                Message <span class="required">*</span>
            </label>
            <textarea
                id="messageField"
                name="message"
                placeholder="Write your message here..."
            ></textarea>
            <!-- Error message container for message field -->
            <div class="error-message" id="messageError"></div>
            <!-- Success message container for message field -->
            <div class="success-message" id="messageSuccess"></div>
        </div>

        <!-- Form Buttons -->
        <button type="submit" class="btn-submit" id="submitBtn">Send
Message</button>
        <button type="reset" class="btn-reset" id="resetBtn">Clear
Form</button>
    </form>
  </div>
```

```
<script>
    //
==============================================================
    // Contact Form Validation Script
    //
==============================================================

    // Get form and input field references
    const contactForm = document.getElementById('contactForm');
    const nameField = document.getElementById('nameField');
    const emailField = document.getElementById('emailField');
    const messageField = document.getElementById('messageField');
    const submitBtn = document.getElementById('submitBtn');
    const formSuccess = document.getElementById('formSuccess');

    // Get error message containers
    const nameError = document.getElementById('nameError');
    const emailError = document.getElementById('emailError');
    const messageError = document.getElementById('messageError');

    // Get success message containers
    const nameSuccess = document.getElementById('nameSuccess');
    const emailSuccess = document.getElementById('emailSuccess');
    const messageSuccess = document.getElementById('messageSuccess');

    //
==============================================================
    // Function: Validate Name Field
```

```
// Parameters: nameValue (string)
// Returns: Boolean (true if valid, false if invalid)
// Logic:
//   - Check if name is empty
//   - Check if name has minimum 2 characters
//
==========================================================
function validateName(nameValue) {
    // Trim whitespace from the input
    const trimmedName = nameValue.trim();

    // Check if name is empty
    if (trimmedName === '') {
        showNameError('Name field cannot be empty');
        return false;
    }

    // Check if name has at least 2 characters
    if (trimmedName.length < 2) {
        showNameError('Name must be at least 2 characters long');
        return false;
    }

    // Check if name has at least one letter (not just numbers/special chars)
    if (!/[a-zA-Z]/.test(trimmedName)) {
        showNameError('Name must contain at least one letter');
        return false;
    }
```

```
    // If all validations pass, show success and return true
    showNameSuccess('Name is valid');
    return true;
}

//
==========================================================
// Function: Validate Email Field
// Parameters: emailValue (string)
// Returns: Boolean (true if valid, false if invalid)
// Logic:
//   - Check if email is empty
//   - Check email format using regex pattern
// Regular Expression Breakdown:
//   ^[^\s@]+ - starts with non-whitespace/@ characters
//   @ - must contain @ symbol
//   [^\s@]+ - domain name (non-whitespace/@ characters)
//   \. - must contain dot
//   [^\s@]+ - TLD (top-level domain)
//   $ - end of string
//
==========================================================
function validateEmail(emailValue) {
    // Trim whitespace from the input
    const trimmedEmail = emailValue.trim();

    // Check if email field is empty
```

```
        if (trimmedEmail === '')
            {   showEmailError('Email field cannot be
            empty'); return false;
        }

        // Regular expression pattern for valid email format
        const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        // Test the email against the pattern
        if (!emailPattern.test(trimmedEmail))
            { showEmailError('Please enter a valid email address (e.g.,
user@example.com)');
            return false;
        }

        // If all validations pass, show success and return true
        showEmailSuccess('Email is valid');
        return true;
    }

    //
==========================================================
    // Function: Validate Message Field
    // Parameters: messageValue (string)
    // Returns: Boolean (true if valid, false if invalid)
    // Logic:
    //   - Check if message is empty
    //   - Check if message has minimum 10 characters
```

```
//
==========================================================
function validateMessage(messageValue) {
    // Trim whitespace from the input
    const trimmedMessage = messageValue.trim();

    // Check if message field is empty
    if (trimmedMessage === '') {
        showMessageError('Message field cannot be empty');
        return false;
    }

    // Check if message has at least 10 characters
    if (trimmedMessage.length < 10) {
        showMessageError('Message must be at least 10 characters long');
        return false;
    }

    // If all validations pass, show success and return true
    showMessageSuccess('Message is valid');
    return true;
}

//
==========================================================
// Function: Display name field error message
// Parameters: errorMessage (string)
// Returns: None
```

```
//
==========================================================
function showNameError(errorMessage) {
    // Show error message
    nameError.textContent = errorMessage;
    nameError.classList.add('show');

    // Remove success message if it was showing
    nameSuccess.classList.remove('show');

    // Add error styling to input field
    nameField.classList.add('error');
    nameField.classList.remove('success');
}

//
==========================================================
// Function: Display name field success message
// Parameters: successMessage (string)
// Returns: None
//
==========================================================
function showNameSuccess(successMessage) {
    // Show success message
    nameSuccess.textContent = successMessage;
    nameSuccess.classList.add('show');

    // Remove error message if it was showing
```

```
        nameError.classList.remove('show');

        // Add success styling to input field
        nameField.classList.add('success');
        nameField.classList.remove('error');
    }

    //
==========================================================
    // Function: Display email field error message
    // Parameters: errorMessage (string)
    // Returns: None
    //
==========================================================
    function showEmailError(errorMessage) {
        // Show error message
        emailError.textContent = errorMessage;
        emailError.classList.add('show');

        // Remove success message if it was showing
        emailSuccess.classList.remove('show');

        // Add error styling to input field
        emailField.classList.add('error');
        emailField.classList.remove('success');
    }
```

```
//
==========================================================
// Function: Display email field success message
// Parameters: successMessage (string)
// Returns: None
//
==========================================================
function showEmailSuccess(successMessage) {
    // Show success message
    emailSuccess.textContent = successMessage;
    emailSuccess.classList.add('show');

    // Remove error message if it was showing
    emailError.classList.remove('show');

    // Add success styling to input field
    emailField.classList.add('success');
    emailField.classList.remove('error');
}

//
==========================================================
// Function: Display message field error message
// Parameters: errorMessage (string)
// Returns: None
//
==========================================================
function showMessageError(errorMessage) {
    // Show error message
```

```
        messageError.textContent = errorMessage;
        messageError.classList.add('show');

        // Remove success message if it was showing
        messageSuccess.classList.remove('show');

        // Add error styling to input field
        messageField.classList.add('error');
        messageField.classList.remove('success');
    }

    //
==========================================================
    // Function: Display message field success message
    // Parameters: successMessage (string)
    // Returns: None
    //
==========================================================
    function showMessageSuccess(successMessage) {
        // Show success message
        messageSuccess.textContent = successMessage;
        messageSuccess.classList.add('show');

        // Remove error message if it was showing
        messageError.classList.remove('show');

        // Add success styling to input field
        messageField.classList.add('success');
```

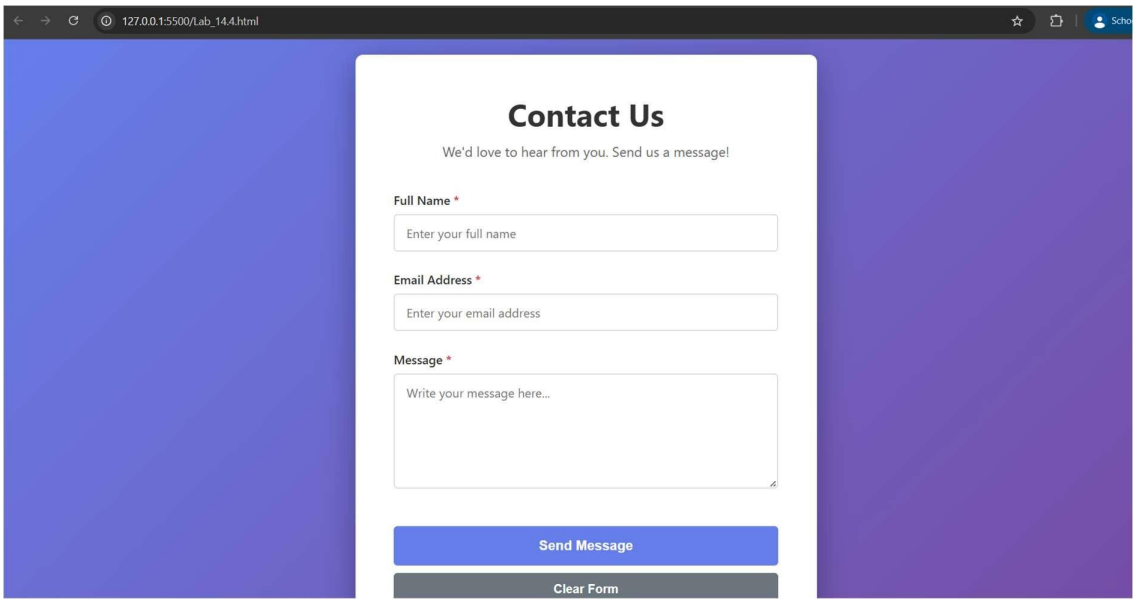
```
        messageField.classList.remove('error');
    }

    //
==============================================================
    // Event Listener: Real-time Name Field Validation
    // Trigger: User types in the name field
    // Action: Validate on each keystroke for immediate feedback
    //
==============================================================
    nameField.addEventListener('input', function() {
        // Call validation function as user types
        validateName(this.value);
    });

    //
==============================================================
    // Event Listener: Real-time Email Field Validation
    // Trigger: User types in the email field
    // Action: Validate on each keystroke for immediate feedback
    //
==============================================================
    emailField.addEventListener('input', function() {
        // Call validation function as user types
        validateEmail(this.value);
    });

    //
==============================================================
```

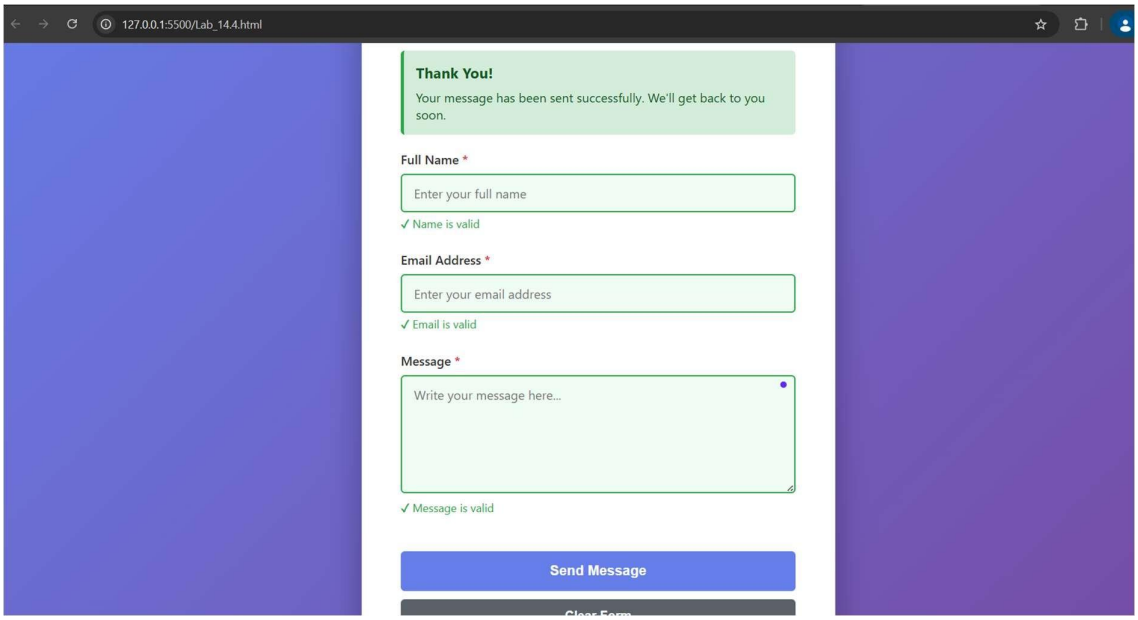
```
// Event Listener: Real-time Message Field Validation
// Trigger: User types in the message field
// Action: Validate on each keystroke for immediate feedback
//
==========================================================
messageField.addEventListener('input', function() {
    // Call validation function as user types
    validateMessage(this.value);
});

//
==========================================================
// Event Listener: Form Submit
// Trigger: User clicks the submit button
// Action: Validate all fields before submission
//
==========================================================
contactForm.addEventListener('submit', function(event) {
    // Prevent default form submission (no page reload)
    event.preventDefault();

    // Validate all three fields
    const isNameValid = validateName(nameField.value);
    const isEmailValid = validateEmail(emailField.value);
    const isMessageValid = validateMessage(messageField.value);

    // Check if all fields are valid
    if (isNameValid && isEmailValid && isMessageValid) {
```

```
        // Show form success message
        formSuccess.classList.add('show');

        // Scroll success message into view
        formSuccess.scrollIntoView({ behavior: 'smooth', block: 'start' });

        // Clear all form fields after successful validation
        contactForm.reset();

        // Remove all success/error styling and messages after 3 seconds
        setTimeout(() => {
            formSuccess.classList.remove('show');
            nameField.classList.remove('success', 'error');
            emailField.classList.remove('success', 'error');
            messageField.classList.remove('success', 'error');
            nameSuccess.classList.remove('show');
            emailSuccess.classList.remove('show');
            messageSuccess.classList.remove('show');
        }, 3000);
    } else {
        // If validation fails, hide success message if shown
        formSuccess.classList.remove('show');
    }
});

// ==========================================================
// Code Summary:
```

```
        // - Real-time validation as user types
        // - Three separate validation functions for each field
        // - Inline error and success messages
        // - Email validation using regex pattern
        // - Minimum length requirements for name and message
        // - Form prevents submission if any field is invalid
        // - No page reload - pure client-side validation
        // - Clear, specific error messages for user guidance
        //
==================================================================
    </script>
</body>
</html>
```

Output:





Explanation:

This is a contact form where users can type their name, email, and message to reach out to a company. As the user types, JavaScript checks if each field is correct it makes sure the name isn't empty, the email has an @ symbol and a dot, and the message is long enough and shows green checkmarks if valid or red error messages if there are mistakes. When the user clicks send, the form checks all fields one final time, and if everything is correct, it shows a success message and clears the form without reloading the page.

Question 4: Dynamic List Management for a Product Page

Prompt:

Generate code for Dynamic List Management for a Product Page. Design a product listing page that needs to add or remove items dynamically without refreshing the page. Create a simple HTML list to display items. Generate javascript that adds a new item when an "Add" Button is clicked, remove the last item when a "Remove" button is clicked. Ensure the DOM updates dynamically. Changes should appeae instantly without page reload. Javascript logic should be clear and maintainable.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Listing - Dynamic Management</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            padding: 20px;
        }

        .container {
            max-width: 800px;
            margin: 0 auto;
            background-color: white;
            border-radius: 10px;
            box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
```

```
    padding: 40px;
}

.header {
    text-align: center;
    margin-bottom: 40px;
}

h1 {
    color: #333;
    margin-bottom: 10px;
    font-size: 2.5rem;
}

.subtitle
    { color:
    #666;
    font-size: 1.1rem;
}

/* Control Section */
.control-section
    { background-color:
    #f8f9fa; padding: 25px;
    border-radius: 8px;
    margin-bottom: 30px;
    border-left: 4px solid #667eea;
}
```

```
.control-section h2
    { color: #333;
    font-size: 1.3rem;
    margin-bottom: 20px;
}

.input-group
    { display: flex;
    gap: 10px;
    margin-bottom: 15px;
    flex-wrap: wrap;
}

input[type="text"]
    { flex: 1;
    min-width: 200px;
    padding: 12px 15px;
    border: 2px solid #e0e0e0;
    border-radius: 5px;
    font-size: 1rem;
    transition: border-color 0.3s ease;
}

input[type="text"]:focus
    { outline: none;
    border-color: #667eea;
```

```
}

.button-group
    {       display:
    flex;       gap:
    10px;
    flex-wrap: wrap;
}

button {
    padding: 12px 25px;
    border: none;
    border-radius: 5px;
    font-size: 1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
}

/* Add Button */
.btn-add {
    background-color: #28a745;
    color: white;
    flex: 1;
    min-width: 120px;
}

.btn-add:hover {
```

```
    background-color: #218838;
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(40, 167, 69, 0.3);
}

.btn-add:active
    {  transform:
    translateY(0);
}

/* Remove Button */
.btn-remove {
    background-color: #dc3545;
    color: white;
    flex: 1;
    min-width: 120px;
}

.btn-remove:hover
    { background-color:
    #c82333; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(220, 53, 69, 0.3);
}

.btn-remove:active
    { transform: translateY(0);
}
```

```
.btn-remove:disabled
    { background-color:
    #ccc; cursor: not-
    allowed; transform:
    none;
}

/* Clear All Button */
.btn-clear {
    background-color: #6c757d;
    color: white;
    flex: 1;
    min-width: 120px;
}

.btn-clear:hover {
    background-color: #5a6268;
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(108, 117, 125, 0.3);
}

/* Product List Section */
.product-section
    { margin-bottom:
    30px;
}

.product-section h2
    { color: #333;
```

```
    font-size: 1.3rem;
    margin-bottom: 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.product-count {
    background-color: #667eea;
    color: white;
    padding: 5px 15px;
    border-radius: 20px;
    font-size: 0.9rem;
    font-weight: bold;
}

/* Product List */
#productList {
    list-style: none;
    padding: 0;
}

.product-item {
    background-color: #f8f9fa;
    padding: 15px;
    margin-bottom: 12px;
    border-radius: 5px;
```

```
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-left: 4px solid #667eea;
    animation: slideIn 0.3s ease;
    transition: all 0.3s ease;
}

.product-item:hover
    { background-color:
    #e9ecef;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.product-info
    { flex: 1;
}

.product-name
    { color: #333;
    font-weight: 600;
    font-size: 1.1rem;
    margin-bottom: 5px;
}

.product-id
    { color: #999;
    font-size: 0.9rem;
```

```
}

.delete-btn {
    background-color: #dc3545;
    color: white;
    padding: 8px 15px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 0.9rem;
    transition: all 0.3s ease;
}

.delete-btn:hover
    { background-color:
    #c82333; transform:
    scale(1.05);
}

/* Empty State */
.empty-state {
    text-align: center;
    padding: 40px 20px;
    color: #999;
}

.empty-state-icon
    { font-size:
    3rem;
```

```
    margin-bottom: 15px;
}

.empty-state-text
    { font-size:
    1.1rem;
}

/* Status Message */
.status-message
    { padding: 15px;
    margin-bottom: 20px;
    border-radius: 5px;
    display: none;
    animation: slideDown 0.3s ease;
}

.status-message.show
    { display: block;
}

.status-message.success
    { background-color:
    #d4edda; color: #155724;
    border-left: 4px solid #28a745;
}

.status-message.error {
```

```
    background-color: #f8d7da;
    color: #721c24;
    border-left: 4px solid #dc3545;
}

/* Animations */
@keyframes slideIn {
    from {
        transform: translateX(-20px);
        opacity: 0;
    }
    to {
        transform: translateX(0);
        opacity: 1;
    }
}

@keyframes slideDown
    { from {
        transform: translateY(-20px);
        opacity: 0;
    }
    to {
        transform: translateY(0);
        opacity: 1;
    }
}
```

```
/* Responsive Design */
@media (max-width: 600px) {
    .container
        { padding:
        25px;
    }

    h1 {
        font-size: 2rem;
    }

    .input-group {
        flex-direction: column;
    }

    input[type="text"]
        { min-width:
        100%;
    }

    .button-group {
        flex-direction: column;
    }

    button {
        width: 100%;
    }
```

```
            .product-item {
                flex-direction: column;
                align-items: flex-start;
            }

            .delete-btn {
                margin-top: 10px;
                width: 100%;
            }

            .product-section h2
                { flex-direction:
                column; align-items:
                flex-start;
            }

            .product-count
                { margin-top:
                10px;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <!-- Header Section -->
        <div class="header">
            <h1>Product Listing</h1>
            <p class="subtitle">Manage your products dynamically</p>
```

```
    </div>

    <!-- Status Message -->
    <div class="status-message" id="statusMessage"></div>

    <!-- Control Section -->
    <div class="control-section">
        <h2>Add New Product</h2>

        <!-- Input Field -->
        <div class="input-group">
            <input
                type="text"
                id="productInput"
                placeholder="Enter product name (e.g., Laptop, Phone, Tablet...)"
                autocomplete="off"
            >
        </div>

        <!-- Control Buttons -->
        <div class="button-group">
            <button class="btn-add" id="addBtn">+ Add Product</button>
            <button class="btn-remove" id="removeBtn">- Remove
Last</button>
            <button class="btn-clear" id="clearBtn">Clear All</button>
        </div>
    </div>
```

```
        <!-- Product List Section -->
        <div class="product-section">
            <h2>
                Current Products
                <span class="product-count" id="productCount">0 items</span>
            </h2>

            <!-- Product List Container -->
            <ul id="productList">
                <!-- Products will be dynamically added here -->
            </ul>

            <!-- Empty State (shown when no products) -->
             <div class="empty-state" id="emptyState">
                <div class="empty-state-icon"></div>
                <p class="empty-state-text">No products yet. Add one to get
started!</p>
            </div>
        </div>
    </div>

    <script>
        //
==================================================================
        // Dynamic Product List Management Script
        //
==================================================================
```

```
// Get DOM element references
const productInput = document.getElementById('productInput');
const addBtn = document.getElementById('addBtn');
const removeBtn = document.getElementById('removeBtn');
const clearBtn = document.getElementById('clearBtn');
const productList = document.getElementById('productList');
const productCount = document.getElementById('productCount');
const emptyState = document.getElementById('emptyState');
const statusMessage = document.getElementById('statusMessage');

// Array to store all products
// Each product object contains: id, name, and timestamp
let products = [];

// Variable to generate unique product IDs
let productIdCounter = 1;

//
==========================================================
// Function: Display status message
// Parameters: message (string), type (success/error)
// Returns: None
// Purpose: Show feedback messages to user
//
==========================================================
function showMessage(message, type = 'success') {
    // Set the message text content
    statusMessage.textContent = message;
```

```
    // Remove existing type classes
    statusMessage.classList.remove('success', 'error');

    // Add the specified type class
    statusMessage.classList.add(type);

    // Show the message by adding 'show' class
    statusMessage.classList.add('show');

    // Auto-hide message after 3 seconds
    setTimeout(() => {
        statusMessage.classList.remove('show');
    }, 3000);
}

//
==========================================================
// Function: Update product count display
// Parameters: None
// Returns: None
// Purpose: Update the counter showing number of products
//
==========================================================
function updateProductCount() {
    // Get the current number of products
    const count = products.length;
```

```
    // Update the count display
    productCount.textContent = count + (count === 1 ? ' item' : ' items');
}

//
==========================================================
// Function: Toggle empty state visibility
// Parameters: None
// Returns: None
// Purpose: Show/hide empty state based on product count
//
==========================================================
function toggleEmptyState() {
    // Check if product list is empty
    if (products.length === 0) {
        // Show empty state message
        emptyState.style.display = 'block';
        // Hide product list
        productList.style.display = 'none';
    } else {
        // Hide empty state message
        emptyState.style.display = 'none';
        // Show product list
        productList.style.display = 'block';
    }
}
```

```
//
==========================================================
// Function: Render all products to the DOM
// Parameters: None
// Returns: None
// Purpose: Create and display all product items in the list
//
==========================================================
function renderProducts() {
    // Clear the existing list content
    productList.innerHTML = '';

    // Loop through all products in the array
    products.forEach((product, index) => {
        // Create a new list item element
        const li = document.createElement('li');

        // Add the product-item class for styling
        li.className = 'product-item';

        // Create product information HTML
        const productInfo = `
            <div class="product-info">
                <div class="product-name">${product.name}</div>
                <div class="product-id">ID: ${product.id} | Added:
${product.timestamp}</div>
            </div>
        `;
```

```
    // Create delete button
    const deleteBtn = document.createElement('button');
    deleteBtn.className = 'delete-btn';
    deleteBtn.textContent = '✕ Delete';

    // Add delete functionality to individual delete button
    // This allows users to delete specific products
    deleteBtn.addEventListener('click', function() {
        deleteProduct(index);
    });

    // Set the HTML content of the list item
    li.innerHTML = productInfo;

    // Append the delete button to the list item
    li.appendChild(deleteBtn);

    // Add the list item to the product list
    productList.appendChild(li);
});

// Update the product count display
updateProductCount();

// Toggle empty state visibility
toggleEmptyState();
```

```
}

//
==========================================================
// Function: Add a new product to the list
// Parameters: None
// Returns: None
// Purpose: Validate input, create product object, and add to DOM
//
==========================================================
function addProduct() {
    // Get the input value and trim whitespace
    const productName = productInput.value.trim();

    // Validate: Check if input is empty
    if (productName === '') {
        // Show error message
        showMessage('Please enter a product name!', 'error');
        // Focus the input field for user convenience
        productInput.focus();
        return; // Exit function early
    }

    // Validate: Check if product name is at least 2 characters
    if (productName.length < 2) {
        showMessage('Product name must be at least 2 characters!', 'error');
        productInput.focus();
        return;
```

```
        }

        // Create a new product object with unique ID and timestamp
        const newProduct = {
            id: productIdCounter,
            name: productName,
            timestamp: new Date().toLocaleTimeString()
        };

        // Add the new product to the products array
        products.push(newProduct);

        // Increment the product ID counter for next product
        productIdCounter++;

        // Clear the input field after adding product
        productInput.value = '';

        // Render all products to update the DOM
        renderProducts();

        // Show success message
        showMessage(`"${productName}" has been added successfully!`,
'success');

        // Return focus to input field for continuous adding
        productInput.focus();
    }
```

```
//
==========================================================
// Function: Remove the last product from the list
// Parameters: None
// Returns: None
// Purpose: Remove most recently added product
//
==========================================================
function removeLastProduct() {
    // Check if there are any products to remove
    if (products.length === 0) {
        // Show error message
        showMessage('No products to remove!', 'error');
        return; // Exit function early
    }

    // Get the last product before removing
    const removedProduct = products[products.length - 1];

    // Remove the last product from the array
    // pop() removes and returns the last element
    products.pop();

    // Render all products to update the DOM
    renderProducts();

    // Show success message with removed product name
```

```
        showMessage(`"${removedProduct.name}" has been removed!`,
'success');
    }

    //
==========================================================
    // Function: Delete a specific product by index
    // Parameters: index (number) - position in products array
    // Returns: None
    // Purpose: Remove a product at a specific position
    //
==========================================================
    function deleteProduct(index) {
        // Validate: Check if index is within array bounds
        if (index < 0 || index >= products.length) {
            showMessage('Error: Product not found!', 'error');
            return;
        }

        // Get the product being deleted
        const deletedProduct = products[index];

        // Remove the product at the specified index
        // splice() removes elements from an array and returns removed elements
        products.splice(index, 1);

        // Render all products to update the DOM
        renderProducts();
```

```
        // Show success message
        showMessage(`"${deletedProduct.name}" has been deleted!`, 'success');
    }

    //
==========================================================
    // Function: Clear all products from the list
    // Parameters: None
    // Returns: None
    // Purpose: Remove all products with confirmation
    //
==========================================================
    function clearAllProducts() {
        // Ask user for confirmation before clearing all products
        if (products.length === 0) {
            showMessage('The list is already empty!', 'error');
            return;
        }

        // Show confirmation dialog
        if (confirm('Are you sure you want to delete all products? This cannot be
undone.')) {
            // Get the count before clearing
            const count = products.length;

            // Clear the products array
            products = [];
```

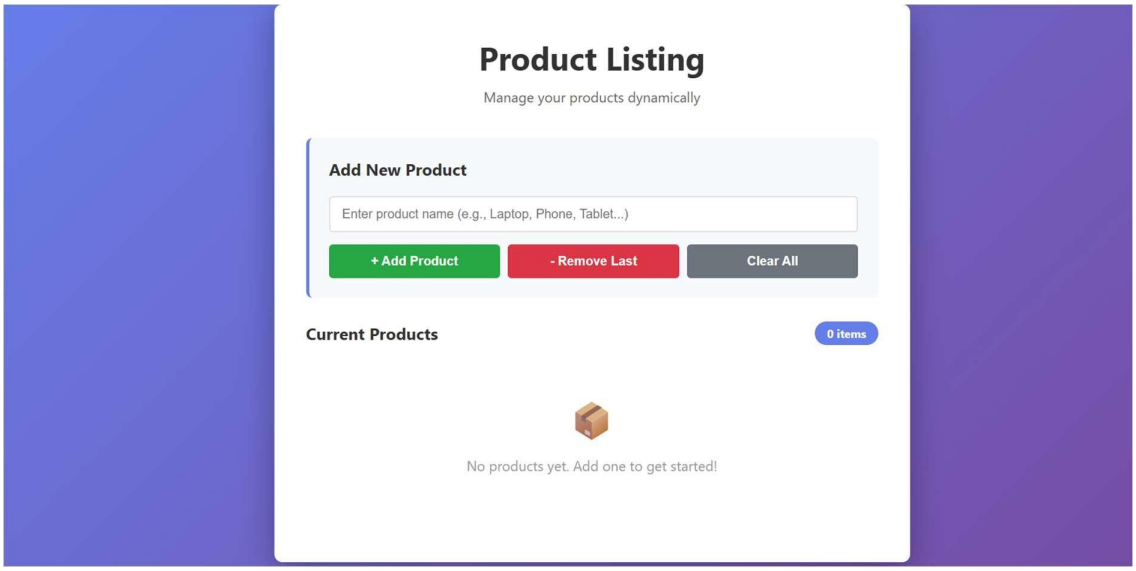
```
            // Reset the product ID counter
            productIdCounter = 1;

            // Render to update DOM (will show empty state)
            renderProducts();

            // Show success message
            showMessage(`All ${count} product(s) have been cleared!`,
'success');

            // Clear the input field
            productInput.value = '';

            // Focus the input field
            productInput.focus();
        }
    }

    //
==========================================================
    // Event Listener: Add Button Click
    // Trigger: User clicks "Add Product" button
    // Action: Call addProduct function
    //
==========================================================
    addBtn.addEventListener('click', function()
        { addProduct();
```

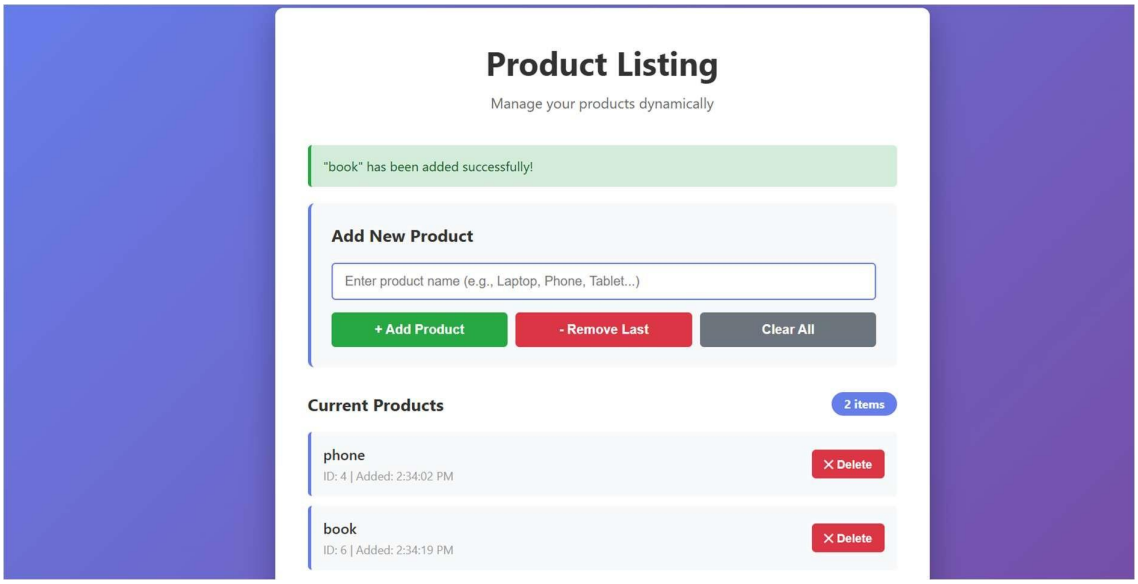
```
});

//
==========================================================
// Event Listener: Remove Button Click
// Trigger: User clicks "Remove Last" button
// Action: Call removeLastProduct function
//
==========================================================
removeBtn.addEventListener('click', function()
    { removeLastProduct();
});

//
==========================================================
// Event Listener: Clear Button Click
// Trigger: User clicks "Clear All" button
// Action: Call clearAllProducts function with confirmation
//
==========================================================
clearBtn.addEventListener('click', function()
    { clearAllProducts();
});

//
==========================================================
// Event Listener: Input Field Enter Key
// Trigger: User presses Enter in the product input field
// Action: Add product (same as clicking Add button)
```

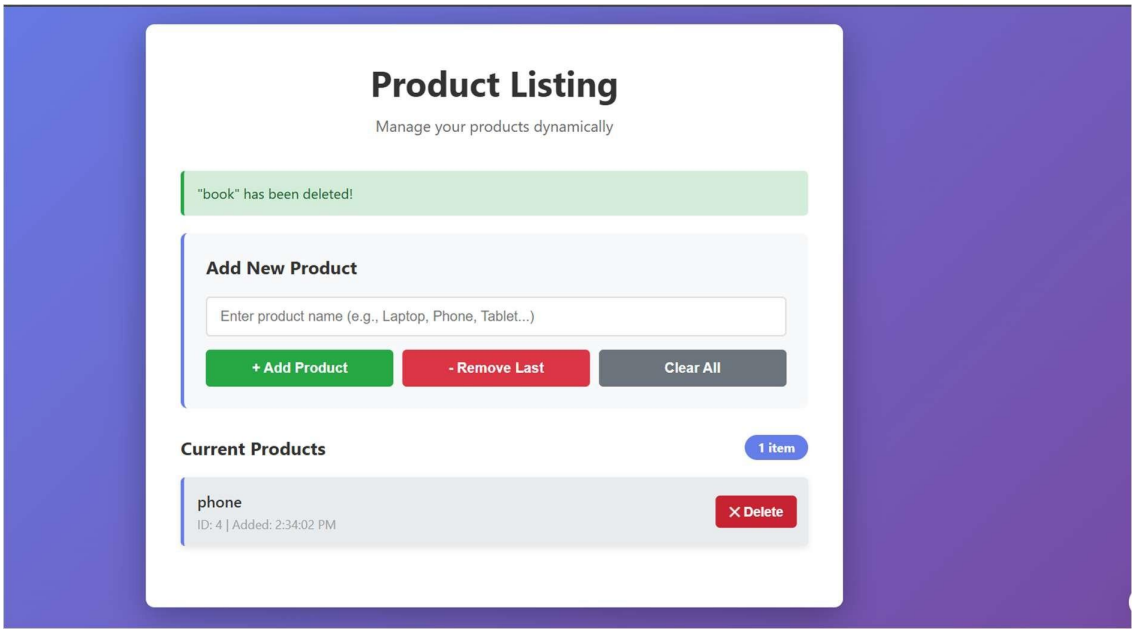
```
// Purpose: Improve UX - allow quick adding with keyboard
//
==========================================================
productInput.addEventListener('keypress', function(event) {
    // Check if the pressed key is Enter (keyCode 13)
    if (event.key === 'Enter') {
        // Call addProduct function
        addProduct();
    }
});

//
==========================================================
// Initialize: Load initial products on page load
// Trigger: When page first loads
// Action: Display empty state
//
==========================================================
renderProducts();
productInput.focus();

//
==========================================================
// Code Summary:
// - Products stored in array for easy management
// - Add function validates input before adding
// - Remove last function removes most recent product
// - Delete individual function allows targeted removal
// - Clear all function asks for confirmation
```

```
    // - DOM updates instantly without page reload
    // - Real-time product count display
    // - Empty state shown when no products exist
    // - Status messages provide user feedback
    // - Enter key supports quick product adding
    //
==========================================================================

  </script>
</body>
</html>
```

Output:







Explanation:

This page lets you manage a product list where you can type a product name and click "Add" to add it to a list, or click "Remove Last" to delete the most recently added product. You can also click a delete button next to any product to remove just that one, and each action shows a success message and updates the count instantly without reloading the page. The page keeps track of all your products with their IDs and timestamps, and shows an empty state message when there are no products.

Question 5: Modal Popup for User Notifications

Prompt:

Generate a code for Modal Popup for User Notifications. Build a user notification popup for announcements or alerts on a website. Generate HTML Structure for a modal popup. Generate CSS styling with a semi transparent background overlay. Generate javascript to open and close the modal. Include multiple ways to close the modal. Clicking "Open Modal" displays the popup, popup overlays the page content clearly, modal closes smoothly when instructed.

Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Modal Popup Notifications</title>

    <style>

        * {

            margin: 0;

            padding: 0;

            box-sizing: border-box;

        }


        body {

            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);

            min-height: 100vh;

            padding: 20px;

        }
```

```
.container {
    max-width: 900px;
    margin: 0 auto;
    background-color: white;
    border-radius: 10px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
    padding: 50px;
}

.header {
    text-align: center;
    margin-bottom: 50px;
}

h1 {
    color: #333;
    margin-bottom: 10px;
    font-size: 2.5rem;
}

.subtitle
    { color:
    #666;
    font-size: 1.1rem;
}

/* Main Content Section */
```

```
.content-section
    { margin-bottom:
    40px;
}

.section-title
    { color: #333;
    font-size: 1.5rem;
    margin-bottom: 20px;
    padding-bottom: 10px;
    border-bottom: 2px solid #667eea;
}

.button-grid
    { display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 15px;
    margin-bottom: 30px;
}

.btn-modal {
    padding: 15px 20px;
    border: none;
    border-radius: 5px;
    font-size: 1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
```

```
}

/* Button Styles */
.btn-success {
    background-color: #28a745;
    color: white;
}

.btn-success:hover
    { background-color:
    #218838; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(40, 167, 69, 0.3);
}

.btn-error {
    background-color: #dc3545;
    color: white;
}

.btn-error:hover {
    background-color: #c82333;
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(220, 53, 69, 0.3);
}

.btn-warning {
    background-color: #ffc107;
```

```
    color: #333;
}

.btn-warning:hover
    { background-color:
    #e0a800; transform:
    translateY(-2px);
    box-shadow: 0 5px 15px rgba(255, 193, 7, 0.3);
}

.btn-info {
    background-color: #17a2b8;
    color: white;
}

.btn-info:hover {
    background-color: #138496;
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(23, 162, 184, 0.3);
}

.btn-modal:active
    { transform: translateY(0);
}

/* Modal Styling */
.modal-overlay
    { display: none;
```

```
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    z-index: 1000;
    justify-content: center;
    align-items: center;
    animation: fadeIn 0.3s ease;
}

.modal-overlay.show
    { display: flex;
}

.modal-overlay.hide { animation:
    fadeOut 0.3s ease;
}

/* Modal Container */
.modal-container
    { background-color:
    white; border-radius:
    10px;
    box-shadow: 0 10px 50px rgba(0, 0, 0, 0.3);
    max-width: 500px;
    width: 90%;
```

```
    animation: slideUp 0.3s ease;
    overflow: hidden;
}

.modal-overlay.hide .modal-container
    { animation: slideDown 0.3s ease;
}

/* Modal Header */
.modal-header
    { padding: 25px;
    border-bottom: 2px solid #f0f0f0;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.modal-title {
    font-size:  1.5rem;
    font-weight:  bold;
    color: #333;
    margin: 0;
}

.modal-icon {
    font-size: 1.5rem;
    margin-right: 10px;
```

```
}

/* Close Button (X) */
.modal-close
    { background: none;
    border: none;
    font-size: 1.5rem;
    color: #999;
    cursor: pointer;
    transition: color 0.3s ease;
    padding: 5px 10px;
}

.modal-close:hover
    { color: #333;
}

/* Modal Body */
.modal-body
    { padding: 25px;
    color: #666;
    line-height: 1.6;
    max-height: 400px;
    overflow-y: auto;
}

.modal-body p {
```

```
    margin-bottom: 15px;
}

/* Modal Footer */
.modal-footer
    { padding: 20px
    25px;
    border-top: 1px solid #f0f0f0;
    display: flex;
    justify-content: flex-end;
    gap: 10px;
    background-color: #f8f9fa;
}

.btn-modal-action
    { padding: 10px
    25px; border: none;
    border-radius: 5px;
    font-size: 1rem;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
}

.btn-close-modal
    { background-color:
    #6c757d; color: white;
}
```

```
.btn-close-modal:hover
    { background-color:
    #5a6268;
}

.btn-confirm {
    background-color: #667eea;
    color: white;
}

.btn-confirm:hover
    { background-color:
    #5568d3;
}

/* Modal Type Styles */
.modal-success .modal-header
    { background-color: #d4edda;
    border-bottom-color: #28a745;
}

.modal-success .modal-title
    { color: #155724;
}

.modal-error .modal-header
    { background-color: #f8d7da;
    border-bottom-color: #dc3545;
```

```
}

.modal-error .modal-title
    { color: #721c24;
}

.modal-warning .modal-header
    { background-color: #fff3cd;
    border-bottom-color: #ffc107;
}

.modal-warning .modal-title
    { color: #856404;
}

.modal-info .modal-header
    { background-color: #d1ecf1;
    border-bottom-color: #17a2b8;
}

.modal-info .modal-title
    { color: #0c5460;
}

/* Animations */
@keyframes fadeIn {
    from {
```

```
        opacity: 0;
    }
    to {
        opacity: 1;
    }
}

@keyframes fadeOut
    { from {
        opacity: 1;
    }
    to {
        opacity: 0;
    }
}

@keyframes slideUp
    { from {
        transform: translateY(30px);
        opacity: 0;
    }
    to {
        transform: translateY(0);
        opacity: 1;
    }
}
```

```
@keyframes slideDown
    { from {
        transform: translateY(0);
        opacity: 1;
    }
    to {
        transform: translateY(30px);
        opacity: 0;
    }
}

/* Info Box */
.info-box {
    background-color: #f0f8ff;
    border-left: 4px solid #17a2b8;
    padding: 15px;
    border-radius: 5px;
    margin-bottom: 30px;
    color: #0c5460;
}

.info-box h3 {
    margin-bottom: 10px;
    color: #0c5460;
}

/* Responsive Design */
```

```
        @media (max-width: 600px) {
            .container
                { padding:
                30px;
            }

            h1 {
                font-size: 1.8rem;
            }

            .button-grid {
                grid-template-columns: 1fr;
            }

            .modal-container
                { width: 95%;
            }

            .modal-footer {
                flex-direction: column-reverse;
            }

            .btn-modal-action
                { width: 100%;
            }
        }
    </style>
</head>
```

```
<body>
    <div class="container">
        <!-- Header Section -->
        <div class="header">
            <h1>Modal Popup Notifications</h1>
            <p class="subtitle">Click buttons to open different types of modal
popups</p>
        </div>

        <!-- Info Box -->
        <div class="info-box">
            <h3>How to Close the Modal:</h3>
            <ul>
                <li>Click the <strong>✕ button</strong> in the top-right corner</li>
                <li>Click the <strong>"Close" button</strong> in the footer</li>
                <li>Click outside the modal on the <strong>dark
overlay</strong></li>
                <li>Press the <strong>ESC key</strong> on your keyboard</li>
            </ul>
        </div>

        <!-- Notification Types Section -->
        <div class="content-section">
            <h2 class="section-title">Notification Types</h2>
            <div class="button-grid">
                <button class="btn-modal btn-success" id="openSuccessModal">
                    ✓ Success Modal
                </button>
```

```
            <button class="btn-modal btn-error" id="openErrorModal">
                ✗ Error Modal
            </button>
            <button class="btn-modal btn-warning" id="openWarningModal">
                ⚠ Warning Modal
            </button>
            <button class="btn-modal btn-info" id="openInfoModal">
                ℹ Info Modal
            </button>
        </div>
    </div>

    <!-- Custom Message Section -->
    <div class="content-section">
        <h2 class="section-title">Custom Modal</h2>
        <div class="button-grid">
            <button class="btn-modal btn-info" id="openCustomModal">
                🔍 Custom Modal
            </button>
        </div>
    </div>
</div>

<!-- ==========================================
     MODAL POPUPS
     ========================================== -->
```

```
<!-- Success Modal -->
<div class="modal-overlay" id="successModal">
    <div class="modal-container modal-success">
      <div class="modal-header">
        <div>
          <span class="modal-icon">✓</span>
           <h2 class="modal-title">Success!</h2>
        </div>
        <button class="modal-close" data-modal="successModal">✕</button>
      </div>
      <div class="modal-body">
        <p>Your operation has been completed successfully! All tasks have been processed without any errors. Thank you for your patience.</p>
      </div>
      <div class="modal-footer">
        <button class="btn-modal-action btn-close-modal" data-modal="successModal">Close</button>
        <button class="btn-modal-action btn-confirm">Confirm</button>
      </div>
    </div>
</div>


<!-- Error Modal -->
<div class="modal-overlay" id="errorModal">
    <div class="modal-container modal-error">
      <div class="modal-header">
        <div>
```

```
            <span class="modal-icon">✗</span>
            <h2 class="modal-title">Error!</h2>
          </div>
          <button class="modal-close" data-modal="errorModal">✕</button>
        </div>
        <div class="modal-body">
          <p>An error occurred while processing your request. Please check your input and try again. If the problem persists, contact our support team.</p>
        </div>
        <div class="modal-footer">
          <button class="btn-modal-action btn-close-modal" data-modal="errorModal">Close</button>
          <button class="btn-modal-action btn-confirm">Retry</button>
        </div>
      </div>
    </div>

    <!-- Warning Modal -->
    <div class="modal-overlay" id="warningModal">
      <div class="modal-container modal-warning">
        <div class="modal-header">
          <div>
            <span class="modal-icon">⚠</span>
            <h2 class="modal-title">Warning!</h2>
          </div>
          <button class="modal-close" data-modal="warningModal">✕</button>
        </div>
```

```
        <div class="modal-body">
            <p>Please be aware that this action may have consequences. Make sure you understand the implications before proceeding. This warning cannot be dismissed lightly.</p>
        </div>
        <div class="modal-footer">
            <button class="btn-modal-action btn-close-modal" data-modal="warningModal">Dismiss</button>
            <button class="btn-modal-action btn-confirm">Proceed Anyway</button>
        </div>
    </div>
  </div>


  <!-- Info Modal -->
  <div class="modal-overlay" id="infoModal">
    <div class="modal-container modal-info">
        <div class="modal-header">
            <div>
                <span class="modal-icon">i</span>
                <h2 class="modal-title">Information</h2>
            </div>
            <button class="modal-close" data-modal="infoModal">X</button>
        </div>
        <div class="modal-body">
            <p>This is an informational message. You can close this modal at any time by clicking the X button, the close button below, clicking outside the modal, or pressing the ESC key.</p>
        </div>
```

```
        <div class="modal-footer">
            <button class="btn-modal-action btn-close-modal" data-modal="infoModal">Got it!</button>
        </div>
    </div>
</div>

<!-- Custom Modal -->
<div class="modal-overlay" id="customModal">
    <div class="modal-container modal-info">
        <div class="modal-header">
            <div>
                <span class="modal-icon"> </span>
                <h2 class="modal-title">Custom Notification</h2>
            </div>
            <button class="modal-close" data-modal="customModal">×</button>
        </div>
        <div class="modal-body">
            <p><strong>Welcome to the Custom Modal!</strong></p>
            <p>This modal demonstrates how you can create custom notifications for any purpose. You have full control over the content, styling, and behavior.</p>
            <p>Try closing this modal using any of the available methods:</p>
            <ul style="margin-left: 20px;">
                <li>Click the × button</li>
                <li>Click the Close button below</li>
                <li>Click the dark background</li>
```

```
                <li>Press ESC key</li>
            </ul>
        </div>
        <div class="modal-footer">
            <button class="btn-modal-action btn-close-modal" data-
modal="customModal">Close</button>
        </div>
    </div>
</div>

<script>
    //
==========================================================
    // Modal Popup Management Script
    //
==========================================================

    // Get DOM element references for modal open buttons
    const openSuccessBtn = document.getElementById('openSuccessModal');
    const openErrorBtn = document.getElementById('openErrorModal');
    const openWarningBtn =
document.getElementById('openWarningModal');
    const openInfoBtn = document.getElementById('openInfoModal');
    const openCustomBtn = document.getElementById('openCustomModal');

    // Get all modal overlays by ID
    const successModal = document.getElementById('successModal');
    const errorModal = document.getElementById('errorModal');
    const warningModal = document.getElementById('warningModal');
```

```
const infoModal = document.getElementById('infoModal');
const customModal = document.getElementById('customModal');

// Store all modals in an object for easier management
const modals = {
    'successModal': successModal,
    'errorModal': errorModal,
    'warningModal': warningModal,
    'infoModal': infoModal,
    'customModal': customModal
};

//
==========================================================
// Function: Open Modal
// Parameters: modalId (string) - ID of modal to open
// Returns: None
// Purpose: Display modal with smooth animation
//
==========================================================
function openModal(modalId) {
    // Get the modal element by ID
    const modal = document.getElementById(modalId);

    // Check if modal exists
    if (!modal) {
        console.error(`Modal with ID "${modalId}" not found`);
        return;
```

```
        }

        // Remove 'hide' class if it exists from previous close animation
        modal.classList.remove('hide');

        // Add 'show' class to display the modal
        modal.classList.add('show');

        // Prevent body scroll when modal is open
        document.body.style.overflow = 'hidden';
    }

    //
==========================================================
    // Function: Close Modal
    // Parameters: modalId (string) - ID of modal to close
    // Returns: None
    // Purpose: Hide modal with smooth animation
    //
==========================================================
    function closeModal(modalId) {
        // Get the modal element by ID
        const modal = document.getElementById(modalId);

        // Check if modal exists
        if (!modal) {
            console.error(`Modal with ID "${modalId}" not found`);
            return;
```

```
        }

        // Add 'hide' class for closing animation
        modal.classList.add('hide');

        // Remove 'show' class to hide the modal
        modal.classList.remove('show');

        // Allow body scroll again after closing
        document.body.style.overflow = 'auto';

        // Wait for animation to complete before fully removing classes
        setTimeout(() => {
            modal.classList.remove('hide');
        }, 300); // Match animation duration
    }

    //
==========================================================
    // Event Listeners: Open Modal Buttons
    // Trigger: User clicks button to open specific modal
    // Action: Call openModal function with modal ID
    //
==========================================================
    openSuccessBtn.addEventListener('click', () =>
openModal('successModal'));
    openErrorBtn.addEventListener('click', () => openModal('errorModal'));
    openWarningBtn.addEventListener('click', () =>
openModal('warningModal'));
```

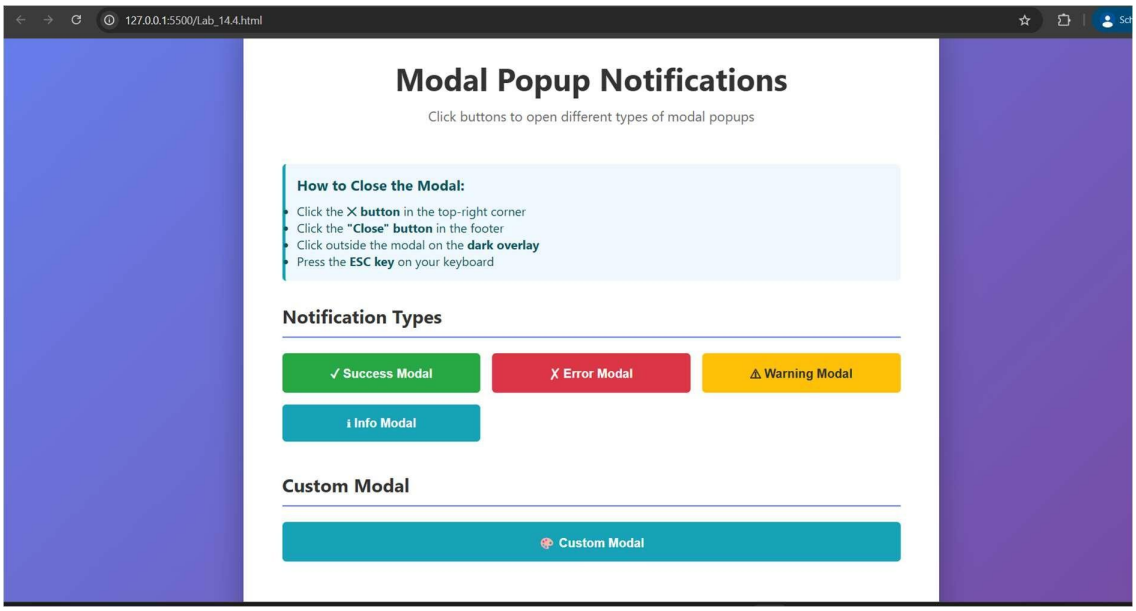
```
openInfoBtn.addEventListener('click', () => openModal('infoModal'));
openCustomBtn.addEventListener('click', () =>
openModal('customModal'));


//
==============================================================
// Event Listeners: Close Buttons (X)
// Trigger: User clicks X button in modal header
// Action: Call closeModal function
// Purpose: Provide easy close method in header
//
==============================================================
document.querySelectorAll('.modal-close').forEach(button =>
  { button.addEventListener('click', function() {
    // Get the modal ID from data attribute
    const modalId = this.getAttribute('data-modal');
    // Call closeModal function with the modal ID
    closeModal(modalId);
  });
});


//
==============================================================
// Event Listeners: Close Buttons (footer)
// Trigger: User clicks "Close" button in modal footer
// Action: Call closeModal function
// Purpose: Provide secondary close button in footer
//
==============================================================
```

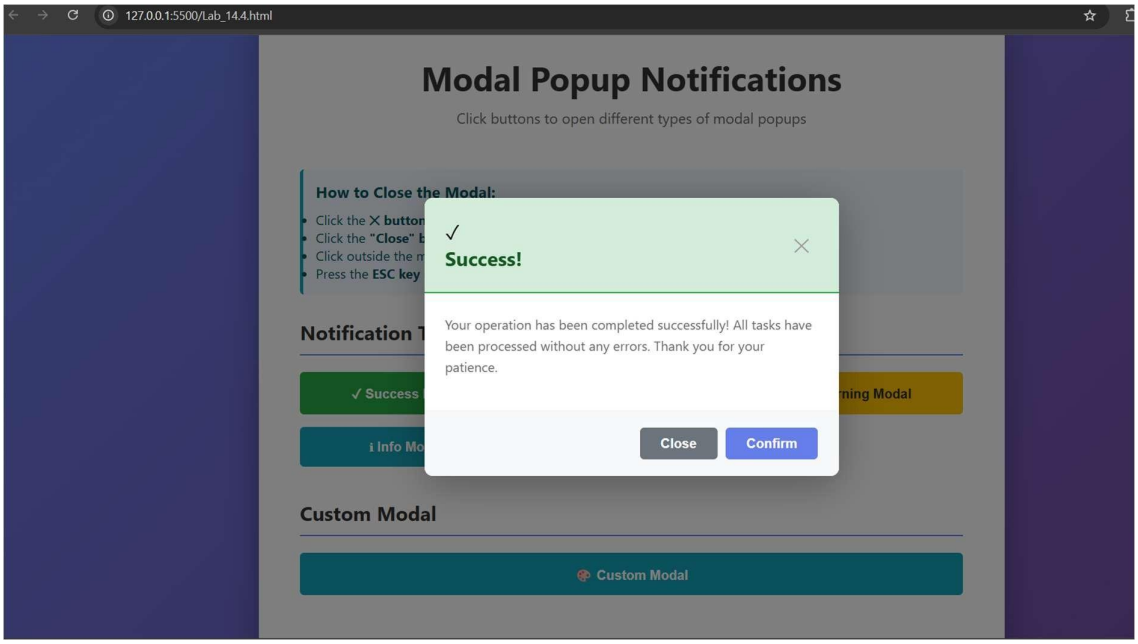
```
document.querySelectorAll('.btn-close-modal').forEach(button =>
    { button.addEventListener('click', function() {
        // Get the modal ID from data attribute
        const modalId = this.getAttribute('data-modal');
        // Call closeModal function with the modal ID
        closeModal(modalId);
    });
});

//
==============================================================
// Event Listener: Overlay Click (Backdrop Click)
// Trigger: User clicks on the dark overlay background
// Action: Close the modal
// Purpose: Allow closing by clicking outside the modal
//
==============================================================
document.querySelectorAll('.modal-overlay').forEach(overlay =>
    { overlay.addEventListener('click', function(event) {
        // Check if the click was on the overlay itself, not the modal container
        if (event.target === this) {
            // Get the modal ID from the overlay's ID attribute
            const modalId = this.id;
            // Call closeModal function
            closeModal(modalId);
        }
    });
});
```

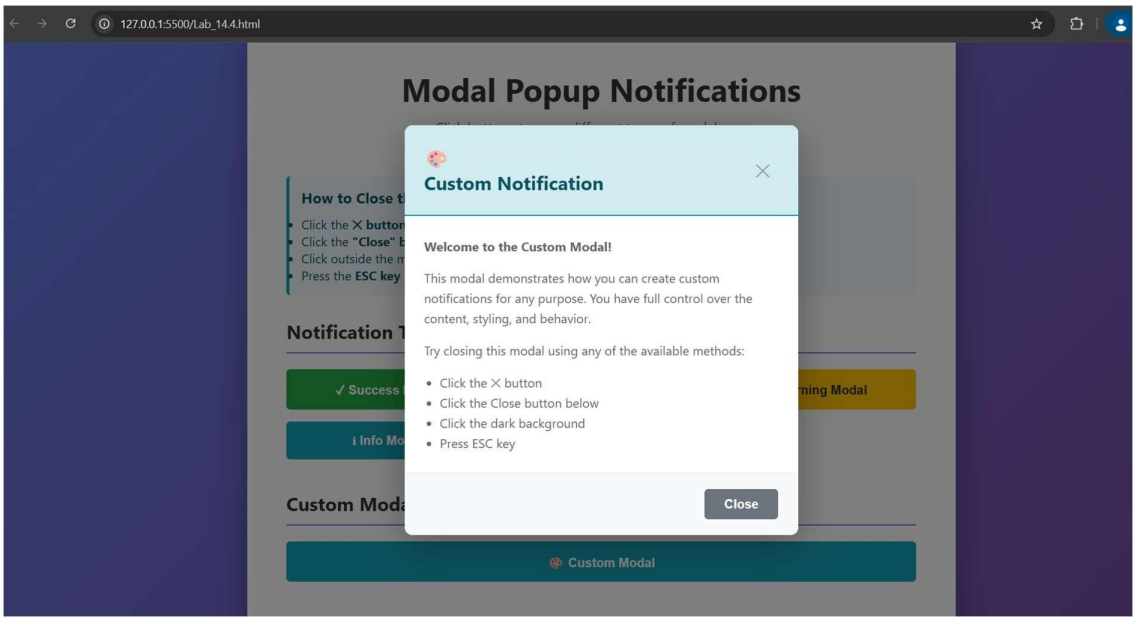
```
//
==========================================================
// Event Listener: ESC Key Press
// Trigger: User presses the ESC key on keyboard
// Action: Close currently open modal
// Purpose: Standard keyboard shortcut for closing modals
//
==========================================================
document.addEventListener('keydown', function(event) {
    // Check if the pressed key is Escape (keyCode 27)
    if (event.key === 'Escape') {
        // Loop through all modals to find which one is open
        Object.keys(modals).forEach(modalId => {
            // Check if modal has 'show' class (is currently open)
            if (modals[modalId].classList.contains('show')) {
                // Close the modal
                closeModal(modalId);
            }
        });
    }
});

//
==========================================================
// Code Summary:
// - Multiple ways to close: X button, Close button, backdrop, ESC key
// - Smooth animations for opening and closing
```

```
        // - Color-coded modal types: success, error, warning, info
        // - Prevents body scroll when modal is open
        // - All modals use same open/close functions
        // - Responsive design for mobile devices
        // - Clear visual feedback for user interactions
        //
==========================================================================
    </script>
</body>
</html>
```

Output:







Explanation:

This page has colorful buttons that open popup windows (modals) with different messages like success, error, warning, and information notifications. You can close each popup in four different ways by clicking the X button, clicking a Close button, clicking the dark area outside the popup, or pressing the ESC key on your keyboard. The popups smoothly slide up when they open and slide down when they close, and the dark background overlay makes it clear the popup is a separate dialog on top of the page.